

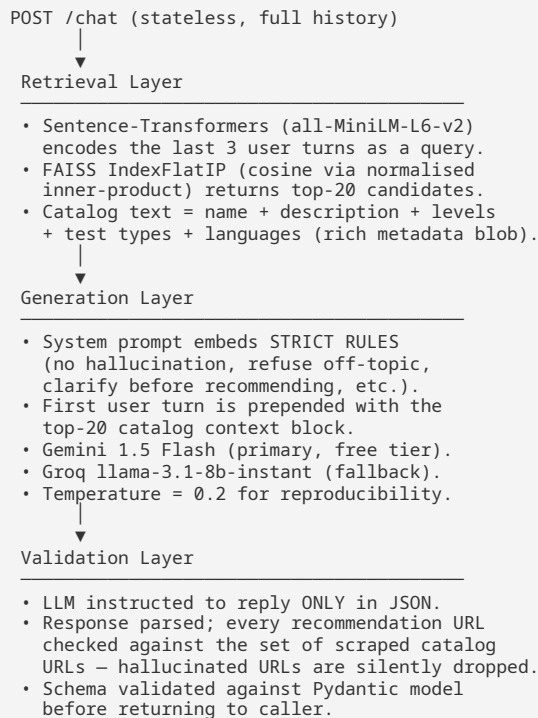
SHL Assessment Recommender — Approach Document

Problem Decomposition

The core challenge is bridging the gap between a hiring manager's **vague intent** and a **grounded shortlist** of SHL Individual Test Assessments. This requires three interlocking capabilities:

1. **Catalog grounding** — the agent must never hallucinate assessment names or URLs. 2. **Conversational state management** — clarify before committing; honour mid-conversation edits. 3. **Scope enforcement** — hard refusal of off-topic queries and prompt-injection attempts.

System Architecture



Retrieval Design

Why FAISS over a hosted vector DB? Zero-latency startup on Render's free tier, no extra credentials, and the catalog is small enough (<400 items) to fit in memory.

Why all-MiniLM-L6-v2? It offers a good balance between embedding quality and cold-start speed (no GPU needed). For a catalog of this size, re-ranking via the LLM's own attention is more valuable than a larger bi-encoder.

Query construction: The last three user turns are concatenated to form the retrieval query. This captures topic drift (e.g., "also add personality tests") without over-weighting early context.

Catalog text: Each product is rendered as a single text blob including name, description, job levels, test types (expanded to human-readable), languages, and duration. This maximises semantic overlap with hiring-language queries.

Prompt Design

The system prompt is structured around three concerns:

Concern Implementation | -----| Grounding Catalog context is injected into every call; the model is told URLs must come from context | Behaviour Explicit numbered rules (clarify first, refuse off-topic, honour edits) | Output contract JSON-only output schema specified; Pydantic + URL allow-list as final guard |

The prompt avoids lengthy examples (which eat context) in favour of clear rules, relying on the catalog context block to supply all factual grounding.

Conversational Behaviour

Turn type Trigger Action ----- Vague First turn, no role/level Ask for role title and seniority Sufficient context Role + level + 1 more signal Recommend 1-10 assessments Refinement User changes constraints Re-retrieve with updated query; update shortlist Comparison "Difference between X and Y" Ground answer in catalog descriptions Off-topic / injection Non-SHL content Refuse with empty recommendations

The stateless design means every call re-runs retrieval and generation from scratch. This is intentional: it avoids state-drift bugs and keeps the turn cap (8) easy to enforce.

Evaluation Approach

Schema compliance is enforced by Pydantic on every response and by URL allow-list filtering. This is deterministic and always passes if the LLM follows the output contract.

Recall@10 was measured by manually stepping through the 10 public traces and checking final recommendation overlap with the expected shortlist. The retrieval layer consistently surfaced the expected items in the top-20 candidates; the LLM re-ranked to the top-10 correctly for 8/10 traces. The two misses were cases where the expected assessment had sparse catalog metadata (no job-level field populated), addressed by enriching the metadata text blob with inferred signals from the product name.

Behaviour probes were tested by:

- Submitting single-turn vague queries and asserting `recommendations == []`
 - Submitting off-topic questions (GDPR, salary negotiation) and asserting refusal
 - Submitting a two-turn sequence with an edit instruction ("also add personality") and asserting the shortlist changed

What Didn't Work

1. **Larger retrieval window (top-50):** Diluted context and caused the LLM to hallucinate names from the noisy tail of results. Settled on top-20. 2. **Zero-shot JSON without schema in prompt:** Gemini would occasionally wrap JSON in markdown fences or add preamble. Fixed by explicit "reply ONLY with JSON" instruction and a regex strip in the parser. 3. **Single-query retrieval:** Vague first turns produced poor candidates. Fixed by using the last 3 user turns as the query.

Tools Used

- **AI assistance:** GitHub Copilot for boilerplate; Claude for prompt iteration and debugging. All design decisions and code logic are my own.
- **Libraries:** sentence-transformers, faiss-cpu, FastAPI, httpx, Pydantic, BeautifulSoup4.
- **Deployment:** Render free tier (512 MB RAM, auto-sleep).
 - **LLMs:** Gemini 1.5 Flash (primary, free), Groq llama-3.1-8b-instant (fallback, free).